

OpenTeX

Ambiente collaborativo LaTeX self-hosted su MongoDB

Cieri Manuel, Citro Gianluigi

Sommario

OpenTeX è un ambiente collaborativo self-hosted per la scrittura di documenti LaTeX, sviluppato come progetto per il corso di Basi di Dati 2 (NoSQL). Il sistema adotta MongoDB come motore di persistenza, un backend FastAPI/Motor asincrono e un frontend React/Vite, con compilazione dei documenti tramite il motore *Tectonic*. Questa relazione descrive l'installazione, l'architettura, il modello dei dati, le scelte CAP/BASE, le operazioni CRUD, la generazione di dati sintetici, gli indici e i relativi benchmark, e la pipeline di aggregazione (JOIN) realizzata sul progetto, riportando le metriche misurate sul sistema.

Indice

1	Introduzione	2
1.1	Stack tecnologico	2
2	Installazione	2
2.1	Procedura	2
2.2	Servizi esposti	3
3	Architettura	3
3.1	Architettura a tre livelli	3
3.2	Modello dei dati: 5 collezioni	3
3.3	Flusso di autenticazione	4
3.4	Pipeline di compilazione LaTeX	4
4	Modello CAP e proprietà BASE	5
5	API e operazioni CRUD	6
5.1	Controllo accessi (RBAC)	6
6	Generazione di dati sintetici	7
6.1	Output di un'esecuzione reale	7
7	Indici e benchmark	7
7.1	Indici creati	7
7.2	Risultati del benchmark	8
7.3	Uso applicativo dell'indice full-text	8
7.4	Scomposizione del costo della compilazione	8
8	Aggregation pipeline (JOIN)	10
8.1	Esempio di chiamata e risposta	10
9	Metriche di progetto	11
10	Uso dell'intelligenza artificiale	11
11	Conclusioni	12

1 Introduzione

OpenTeX nasce per dimostrare l'uso di un database NoSQL documentale (MongoDB) in un caso d'uso realistico: un editor collaborativo di documenti LaTeX con gestione di utenti, progetti, permessi, file e log di attività. Il progetto è stato sviluppato in maniera incrementale attraverso 14 issue (vedi Sezione 11), ciascuna chiusa con una pull request dedicata e revisionata da un membro del team diverso dall'autore.

1.1 Stack tecnologico

Componente	Tecnologia	Ruolo
Frontend	React 19 + Vite 8	Single-page application
HTTP client	Axios 1.x	Iniezione automatica del Bearer JWT
Editor	@uiw/react-codemirror 4.x	Editor con syntax highlighting LaTeX
Backend	FastAPI + Motor (async)	API REST e logica applicativa
Database	MongoDB 8.0	Persistenza documentale
Compilazione	Tectonic	Compilazione LaTeX → PDF, imbarcato nel backend
Orchestrazione	Docker Compose	3 servizi: mongo, backend, frontend

Tabella 1: Stack tecnologico di OpenTeX.

2 Installazione

Il sistema è interamente containerizzato; l'unico prerequisito è Docker (≥ 24.0) e Docker Compose (≥ 2.20).

2.1 Procedura

1. Clonare il repository e configurare le variabili d'ambiente:

```
git clone <url-repo> && cd opentex
cp .env.example .env # impostare SECRET_KEY, MONGO_ROOT_USER, MONGO_ROOT_PASSWORD
```

2. Avviare tutti i servizi:

```
docker compose up -d --build
```

3. Applicare la validazione dello schema MongoDB (\$jsonSchema):

```
docker compose exec backend python -m scripts.validation.apply_schema_validation
```

4. (Opzionale) Caricare dati sintetici di esempio:

```
python seed/seed.py --users 50 --projects 100 --logs 30000 --drop
```

5. Verificare il funzionamento:

```
curl http://localhost:8000/health
```

Servizio	URL	Note
Frontend	http://localhost:5173	React + Vite, hot-reload
Backend API	http://localhost:8000	FastAPI + Motor
Swagger UI	http://localhost:8000/docs	Documentazione API interattiva
MongoDB	localhost:27017	Credenziali in <code>.env</code>

Tabella 2: Servizi e porte esposte da Docker Compose.

2.2 Servizi esposti

Login di test (creato dal seed): `admin@opentex.org` / `password` (account amministratore di sito, con accesso a statistiche e benchmark). Tutti gli account generati dal seed condividono la stessa password; gli indirizzi di esempio sono riportati in `seed/seed_output.txt` dopo ogni esecuzione.

3 Architettura

3.1 Architettura a tre livelli

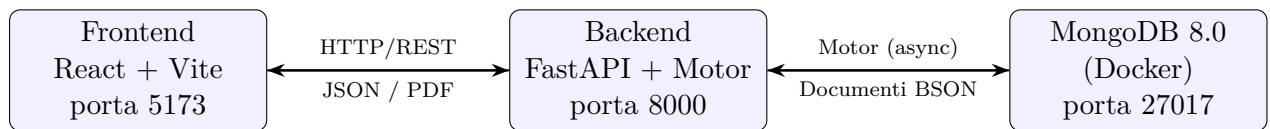


Figura 1: Architettura a tre livelli di OpenTeX. Tutti i servizi sono orchestrati via Docker Compose; il backend imbarca un binario statico di Tectonic, eliminando la necessità di un’installazione TeX separata.

3.2 Modello dei dati: 5 collezioni

Il database è organizzato in cinque collezioni. La scelta tra *embedding* e *referencing* è stata guidata da tre criteri: stabilità del dato, dimensione, e limite di 16 MB per documento imposto da MongoDB.

Decisione	Scelta	Motivazione
Metadati di progetto (tag, abstract)	Embedded	Piccoli, stabili, letti sempre insieme al progetto
File, permessi, log di attività	Referenced (ObjectId)	Crescita illimitata; i file multimediali eccedono il limite di 16 MB/documento
Profili utente	Referenced	Condivisi tra molti progetti; aggiornati indipendentemente

Tabella 3: Criteri di modellazione: embedding vs. referencing.

Collezione	Campi principali
users	email (univoca), first_name, last_name, department, hashed_password, is_admin, created_at, preferences
projects	title, abstract, tags[], owner_id → users, created_at, updated_at
files	project_id → projects, filename, file_type (tex bib image pdf), content (tex/bib) path (image/pdf), uploaded_by → users
permissions	user_id → users, project_id → projects, role (Admin Editor Viewer), granted_at, granted_by
activity_logs	user_id → users, project_id → projects, action, resource, resource_id, timestamp

Tabella 4: Schema sintetico delle 5 collezioni. La validazione `$jsonSchema` applicata in MongoDB impone i campi obbligatori, i tipi e le regole condizionali (es. `content` richiesto per `tex/bib`, `path` per `image/pdf`); non viene invece verificata l'integrità referenziale a livello di database, scelta coerente con il modello AP descritto in Sezione 4.

3.3 Flusso di autenticazione

L'autenticazione è basata su JWT firmati con algoritmo HS256. Le password sono salvate con hash `bcrypt` e non vengono mai restituite in alcuna risposta API. Il token scade dopo `JWT_EXPIRATION_MINUTES` (default 1440 minuti, 24 h).

```
POST /auth/register {email, password, first_name, last_name, department}
-> bcrypt.hash(password) salvato in users; 201 Created

POST /auth/login {email, password}
-> bcrypt.verify; JWT firmato con SECRET_KEY
-> 200 {access_token, token_type, user}

GET /projects/ (Authorization: Bearer <token>)
-> JWT decodificato, user_id iniettato come dipendenza
-> 200 [...]
```

3.4 Pipeline di compilazione LaTeX

La compilazione avviene in una directory temporanea isolata, con timeout e parallelismo limitato per contenere il carico sul backend:

1. Recupero di tutti i file `.tex/.bib` del progetto da MongoDB.
2. Scrittura in una directory temporanea isolata (`tempfile.mkdtemp`).
3. Acquisizione di un semaforo (massimo 2 job paralleli).
4. Esecuzione di `tectonic -keep-logs -outdir <tmp> <entry_point.tex>`, timeout 60 s, `\write18` disabilitato di default.
5. Successo → lettura di `output.pdf`, risposta `application/pdf`.
6. Fallimento → risposta 422 con il log di Tectonic.
7. Timeout → risposta 504.
8. Sempre → `shutil.rmtree(tmp)` (pulizia).

L'entry point è `main.tex` se presente, altrimenti il primo file `.tex` in ordine alfabetico.

Le tre componenti principali del flusso, query al database (passo 1), scrittura su filesystem (passo 2) ed esecuzione del compilatore (passo 4), sono strumentate e misurate separatamente; la relativa analisi è riportata nella Sezione 7.4.

4 Modello CAP e proprietà BASE

OpenTeX gira su una singola istanza/replica MongoDB. Il sistema è progettato secondo il paradigma **AP** del teorema CAP:

- **Availability:** le richieste di lettura e scrittura hanno successo anche in presenza di guasti transitori.
- **Partition Tolerance:** il sistema continua a operare in presenza di partizioni di rete.
- **Consistency** (rilassata): MongoDB garantisce solo atomicità a livello di singolo documento; non esistono transazioni multi-documento o cross-collection. Sono quindi possibili letture stale in casi limite (es. un documento `projects` aggiornato mentre viene letto il relativo `permissions`).

Proprietà BASE	Implementazione in OpenTeX
Basically Available	FastAPI risponde immediatamente; nessun lock globale
Soft state	Gli aggregati derivati (es. conteggio collaboratori) sono calcolati on-demand, non in cache
Eventually consistent	I log di attività e le modifiche ai permessi si propagano alla lettura successiva; nessuna sincronizzazione write-ahead

Tabella 5: Semantica BASE adottata da OpenTeX.

5 API e operazioni CRUD

Il backend espone 7 router FastAPI con operazioni CRUD complete sulle risorse principali, documentate automaticamente via Swagger (/docs).

Router	Metodo	Endpoint	Descrizione
auth	POST	/auth/register	Registrazione utente
	POST	/auth/login	Login, rilascio JWT
projects	GET	/auth/departments	Elenco dipartimenti
	POST	/projects/	Creazione progetto
	GET	/projects/	Elenco progetti (filtri owner_id, member_id; ricerca full-text q)
	GET	/projects/{id}	Dettaglio progetto
	PUT	/projects/{id}	Aggiornamento progetto
	DELETE	/projects/{id}	Cancellazione progetto
	GET	/projects/{id}/files	File collegati al progetto
files	POST	/projects/{id}/files	Creazione file nel progetto
	PUT	/files/{id}	Salvataggio contenuto file
	DELETE	/files/{id}	Cancellazione file
permissions	POST	/projects/{id}/permissions	Assegna/aggiorna ruolo
	DELETE	/projects/{id}/permissions/{uid}	Revoca accesso
	GET	/projects/{id}/permissions	Elenco collaboratori
compile	POST	/projects/{id}/compile	Compilazione LaTeX → PDF
stats	GET	/stats/projects	Statistiche aggregate (JOIN)
	GET	/stats/benchmarks	Risultati benchmark indici
	GET	/stats/compile-benchmark	Ripartizione tempi di compilazione
users	GET	/users/	Elenco utenti
	GET	/users/{id}	Dettaglio utente

Tabella 6: Endpoint REST implementati (21 endpoint su 7 router).

5.1 Controllo accessi (RBAC)

Ruolo	Read	Edit	Delete	Gestione permessi
Admin	✓	✓	✓	✓
Editor	✓	✓	—	—
Viewer	✓	—	—	—

Tabella 7: Ruoli di progetto (collezione permissions). Il proprietario del progetto è sempre Admin implicito ed è l'unico che può cancellarlo. Il flag is_admin a livello account è indipendente da questi ruoli.

6 Generazione di dati sintetici

Per testare query, indici e benchmark in condizioni realistiche è stato sviluppato un generatore di dati sintetici basato su **Faker** (`seed/seed.py`), parametrico nel numero di utenti, progetti e log generati, e con integrità referenziale garantita (ogni `owner_id` punta a un utente esistente, ogni `permission` a una coppia utente-progetto valida).

```
pip install -r seed/requirements.txt
python seed/seed.py --users 50 --projects 100 --logs 30000 --drop
```

6.1 Output di un'esecuzione reale

Collezione	Documenti generati
users	51 (incluso l'account admin)
projects	100
files	337
permissions	251
activity_logs	30 000
Tempo totale	0.3 s

Tabella 8: Risultato di un'esecuzione dello script di seed (`seed/seed_output.txt`).

7 Indici e benchmark

7.1 Indici creati

Indice	Collezione	Campi	Tipo
projects_text_search	projects	title, abstract	Full-text
projects_owner_date	projects	owner_id created_at	+ Composto
permissions_project_id	permissions	project_id	Singolo campo
activity_logs_project_id	activity_logs	project_id	Singolo campo
files_project_id	files	project_id	Singolo campo
email_1	users	email	Univoco

Tabella 9: Indici del sistema. I primi cinque sono creati da `scripts/indexes/create_indexes.py` e verificati con `explain()`. Due di essi sono inoltre garantiti all'avvio dell'applicazione (`backend/app/main.py`). Il primo è un indice univoco su `users.email`, che assolve una duplice funzione: accelera la ricerca dell'utente in fase di login e impone a livello di database l'unicità dell'indirizzo email in fase di registrazione. Il secondo è `projects_text_search`, richiesto dalla ricerca della dashboard: in sua assenza MongoDB rifiuta la query `$text` con errore `IndexNotFound`, condizione che si presenta dopo ogni esecuzione del seed con `-drop`, che elimina le collezioni e con esse i loro indici.

7.2 Risultati del benchmark

Le query sono state misurate con e senza l'indice corrispondente, su 10 esecuzioni per query (media riportata).

Query	Con indice	Senza indice	Speedup
Text search (title + abstract)	1.753 ms	2.003 ms	1.1×
Compound: owner_id + created_at	1.492 ms	1.655 ms	1.1×
Permissions by project_id	1.561 ms	1.667 ms	1.1×
Activity logs by project_id	1.750 ms	19.024 ms	10.9×
Files by project_id	1.609 ms	1.979 ms	1.2×

Tabella 10: Tempi medi di esecuzione su 10 run (`scripts/benchmark/run_benchmark.py`).

Interpretazione. Lo speedup più marcato (10.9×) si osserva sulla query su `activity_logs`, l'unica collezione realmente di grandi dimensioni (30 000 documenti): senza indice MongoDB esegue una scansione completa (`COLLSCAN`) su tutti i 30k documenti, con indice risolve la stessa query in $O(\log n)$ (`IXSCAN`). Le altre query mostrano guadagni modesti (1.1-1.2×) perché le collezioni coinvolte sono piccole (≤ 356 documenti): a questa scala `COLLSCAN` e `IXSCAN` hanno tempi di esecuzione comparabili, e l'overhead della rete Docker domina la misura. Lo speedup di questi indici crescerebbe proporzionalmente al volume di dati.

7.3 Uso applicativo dell'indice full-text

L'indice `projects_text_search` non è un artefatto del solo benchmark: alimenta il campo di ricerca della dashboard, che invoca `GET /projects/?q=<termini>` eventualmente combinato con `owner_id` o `member_id` per restringere la ricerca ai progetti posseduti o condivisi. Il filtro risultante è una query `$text` ordinata per rilevanza (`{ $meta: "textScore" }`). Il piano di esecuzione conferma l'uso dell'indice:

```
FETCH -> TEXT_MATCH -> FETCH -> IXSCAN (indexName: projects_text_search)
```

Si osservi che il filtro su `owner_id` è applicato nello stadio `FETCH` esterno e non risolto dall'indice: MongoDB usa l'indice full-text per risolvere i termini e solo dopo scarta i documenti appartenenti ad altri proprietari. Un indice testuale composto con `owner_id` come chiave prefisso risolverebbe entrambe le condizioni in un'unica scansione, ma MongoDB impone che ogni query su un indice testuale composto specifichi un'uguaglianza su tutte le chiavi prefisso: la ricerca globale, non vincolata a un proprietario, diverrebbe ineseguibile. La scelta dell'indice semplice è quindi un compromesso a favore della generalità della query.

Il limite semantico è che `$text` confronta parole intere (con stemming), non prefissi: `quantum` trova “Quantum computing”, `quant` no. Una ricerca per sottostringa richiederebbe un'espressione regolare non ancorata, che degenera in `COLLSCAN`, esattamente ciò che l'indice serve a evitare, e infatti la stessa equivalenza adottata come baseline “senza indice” in Tabella 10.

7.4 Scomposizione del costo della compilazione

Oltre al confronto con e senza indici, è stato strumentato l'endpoint di compilazione `POST /projects/{id}/compile` per stabilire *dove* si concentri effettivamente il tempo di risposta. Il flusso è stato scomposto in tre fasi misurate separatamente con `time.perf_counter()`:

1. **Query al database** – recupero dei sorgenti del progetto: `find({project_id, file_type: tex|bib})` sulla collezione `files`;
2. **Scrittura su filesystem** – copia dei sorgenti nella directory temporanea isolata;

3. Compilazione LaTeX – esecuzione del subprocess `tectonic`.

Per garantire la riproducibilità della misura, il benchmark compila sempre lo stesso documento: un file `.tex` fisso (`seed/fixtures/benchmark.tex`) inserito dallo script di popolazione in un progetto dedicato. Un documento generato casualmente avrebbe infatti un costo di compilazione variabile, rendendo le medie non confrontabili tra esecuzioni. La metodologia è la stessa adottata per gli indici: media su più esecuzioni con una run di *warm-up* esclusa, necessaria in questo caso perché Tectonic scarica e mette in cache i pacchetti alla prima invocazione.

Fase	Tempo medio (ms)	% del totale
Query al database	2.977	0.69 %
Scrittura su filesystem	0.192	0.04 %
Compilazione LaTeX (Tectonic)	428.834	99.27 %
Totale	432.004	100.00 %

Tabella 11: Ripartizione del tempo di compilazione, media su 5 esecuzioni misurate (una run di warm-up esclusa), documento fisso.

Interpretazione. Il livello dati incide per lo **0.69 %** del tempo totale di risposta: anche azzerando completamente il costo della query, il tempo complessivo passerebbe da 432 a 429 ms, un miglioramento non percepibile dall’utente. Si tratta di un’applicazione diretta della legge di Amdahl: lo speedup ottenibile ottimizzando una componente è limitato superiormente dalla frazione di tempo che tale componente occupa. Il collo di bottiglia è la compilazione vera e propria, che è lavoro di CPU e I/O esterno al database e non comprimibile agendo sul modello dei dati.

È comunque utile osservare che la query di compilazione (2.977 ms) risulta più costosa della corrispondente query sui file misurata in Tabella 10 (1.609 ms), pur insistendo sulla stessa collezione e sullo stesso indice. La differenza è nel tipo di accesso: la seconda è un `count_documents` che si risolve interamente sull’indice, mentre la prima è una `find` che materializza i documenti completi, incluso il campo `content` contenente l’intero sorgente LaTeX. È la distinzione tra la lettura delle sole chiavi di indice e il successivo accesso ai documenti.

Le leve disponibili sul lato database sarebbero due: una *projection* esplicita, per limitare i campi trasferiti a quelli effettivamente utilizzati, e un indice composto `project_id + file_type`, dato che la query filtra su entrambi i campi mentre l’indice attuale copre solo il primo. Entrambe ridurrebbero i 2.977 ms misurati, ma, coerentemente con quanto sopra, l’effetto sul tempo di risposta percepito resterebbe sotto la soglia di misurabilità. La scelta di non implementarle è quindi una conseguenza documentata della misura, non un’omissione.

8 Aggregation pipeline (JOIN)

L'endpoint `GET /stats/projects` realizza una pipeline di aggregazione multi-stadio che unisce quattro collezioni, rappresentando la principale dimostrazione di JOIN cross-collection in MongoDB.

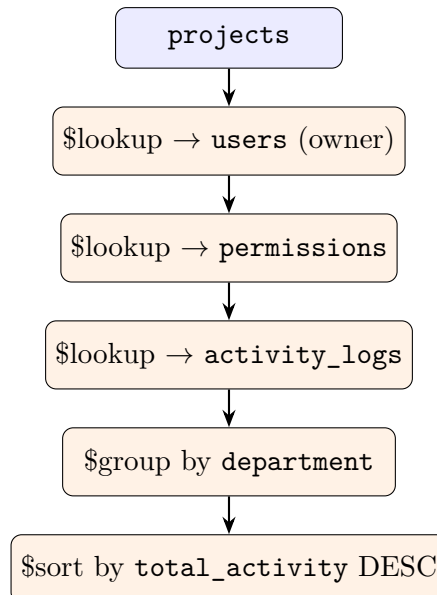


Figura 2: Pipeline di aggregazione di `/stats/projects`. Filtri opzionali (`department`, `date_from`, `date_to`) sono applicati in uno stadio `$match` prima dei lookup.

8.1 Esempio di chiamata e risposta

```
GET /stats/projects?department=Engineering
```

```
[
  {
    "department": "Engineering",
    "project_count": 12,
    "total_collaborators": 34,
    "avg_collaborators": 2.83,
    "total_activity": 1520
  }
]
```

9 Metriche di progetto

Area	Righe di codice (LOC)
Backend Python (<code>backend/</code>)	1 245
Frontend JS/JSX (<code>frontend/src/</code>)	1 466
Script seed + scripts (indici/benchmark/validazione)	1 024
Totale	3 735

Tabella 12: Conteggio righe di codice (`wc -l` sui file `.py/.js/.jsx`, esclusi `node_modules`).

Metrica	Valore
Collezioni MongoDB	5
Indici	6 (5 di query + 1 univoco)
Router API	7
Endpoint REST	21
Ruoli RBAC	3 (Admin / Editor / Viewer)
Pagine frontend	6
Componenti UI condivisi	4
Issue chiuse	14
Pull request mergeate	14

Tabella 13: Metriche riassuntive di progetto.

10 Uso dell'intelligenza artificiale

In conformità con i requisiti del corso, l'uso di strumenti di intelligenza artificiale generativa è dichiarato in modo trasparente. La tracciatura puntuale, issue per issue e PR per PR, è contenuta nei form di valutazione compilati durante lo svolgimento del progetto (FORM 3 e FORM 4). Di seguito una sintesi delle aree in cui gli strumenti sono stati impiegati.

- **Progettazione del modello dati** (Issue #1): supporto al ragionamento sui criteri di embedding e referencing e sulla gestione dei file multimediali, con esplicitazione dei vincoli (limite di 16 MB per documento) alla base delle scelte adottate.
- **Generazione dati sintetici** (Issue #2): supporto alla stesura dello script di seed; output verificato ed eseguito prima del merge.
- **Query e aggregazioni** (Issue #7): supporto alla stesura della pipeline di aggregazione multi-stadio (`$lookup`, `$group`, `$sort`); risultati verificati eseguendo la pipeline sui dati di seed.
- **Indici e benchmark** (Issue #8-#9): supporto alla definizione degli indici, alla lettura dell'output di `explain()` e alla struttura dello script di misura; metodologia e interpretazione dei risultati definite dal team.
- **Autenticazione e compilazione** (Issue #20-#21): supporto alla configurazione dell'hashing con `bcrypt` e dei token JWT, e all'invocazione isolata del compilatore esterno (directory temporanea, timeout, semaforo).

- **Strumentazione della compilazione** (Sezione 7.4): supporto all’implementazione della misurazione per fasi e del relativo endpoint di benchmark; metodologia (documento fisso, warm-up escluso) e interpretazione dei risultati definite dal team.
- **Documentazione** (Issue #10): LLM usato per la generazione della documentazione, il fix del Dockerfile (build-arg TARGETARCH), le traduzioni e il polish del frontend.

In tutti i casi l’output generato è stato eseguito e verificato prima dell’integrazione, e revisionato da un membro del team diverso dall’autore, secondo il workflow standard: 1 issue → 1 branch → 1 PR → 1 review → merge. Nessun contenuto è stato integrato senza essere compreso e verificabile dal team.

11 Conclusioni

Il progetto OpenTeX è stato sviluppato attraverso 14 issue, ciascuna su un branch dedicato e chiusa da una pull request collegata tramite **Closes**. Le prime dieci coprono i requisiti del progetto; le ultime quattro sono estensioni realizzate a partire dai *nice to have* della proposta iniziale.

Issue	Branch	Titolo	PR
<i>Requisiti di progetto</i>			
#1	issue-1-schema-validation	Schema e validation delle 5 collezioni MongoDB	#11
#2	issue-2-seed-data	Generatore di dati sintetici (seed)	#12
#3	issue-3-docker-setup	Setup repository, Docker Compose e configurazione base	#13
#4	issue-4-projects-crud	API CRUD per la gestione dei progetti	#14
#5	issue-5-permissions	Sistema di permessi e gestione collaboratori	#18
#6	issue-6-dashboard-ui	UI Dashboard “I miei Progetti”	#19
#7	issue-7-aggregation	Aggregation pipeline per statistiche (JOIN tra collezioni)	#15
#8	issue-8-indexing	Creazione e ottimizzazione indici	#16
#9	issue-9-benchmark	Benchmark query con e senza indici	#17
#10	issue-10-documentation	Documentazione finale e proof of concept	#28
<i>Estensioni oltre i requisiti minimi</i>			
#20	issue-20-auth	Autenticazione con credenziali e token JWT	#24
#21	issue-21-latex-compiler	Endpoint di compilazione LaTeX lato backend	#25
#22	issue-22-editor-ui	Editor multi-pannello in stile Overleaf	#26
#23	issue-23-pdf-preview	Pannello di anteprima del PDF compilato	#27

Tabella 14: Mappatura completa issue → branch → pull request.

Tutte le issue risultano chiuse e tutte le 14 le pull request mergiate su **main** (l’ultima, PR #28, relativa alla documentazione finale); nessuna modifica è entrata nel branch principale al di fuori di una pull request revisionata. Il sistema è interamente riproducibile da zero seguendo le istruzioni del README, non contiene credenziali reali nel repository, ed è stato verificato end-to-end (registrazione, login, creazione progetto, upload/editing file, compilazione LaTeX, gestione permessi, consultazione statistiche).